

Agent = LLM + 上下文 + 工具——书评李博杰《深入理解AI-Agent：设计原理与工程实践》

书名：《深入理解 AI Agent：设计原理与工程实践》作者：李博杰出版形式：开源电子书（Apache 2.0），GitHub仓库 [bojieli/ai-agent-book](https://github.com/bojieli/ai-agent-book)首次发布：2025年9月GitHub Stars：34,661（截至2026年8月）章节：10章正文 + 引言 + 后记配套实验：95个（含本地项目与外部复现轨道，70+可独立运行）语言：中文原版，13种社区翻译（英/西/印尼/阿拉伯/繁体/俄/泰米尔/越/日/土耳其/韩/匈牙利）在线阅读：<https://bojieli.github.io/ai-agent-book/>PDF下载：GitHub Releases类别：人工智能/AI Agent/工程实践

李博杰，中国科学技术大学与微软亚洲研究院联合培养博士（2019），ACM中国优秀博士学位论文获得者，微软学者。曾在华为2012实验室工作，后离职创业，现任Pine AI首席科学家。Pine AI做的是全自动语音Agent——替用户接电话、谈判。他在SIGCOMM、SOSP、NSDI等顶级会议上发过多篇论文，研究方向从网络系统转向了AI Agent。

这个履历很重要——它解释了这本书为什么和市面上其他AI书籍不一样。李博杰不是自媒体博主，不是培训机构的讲师，不是一个赶风口的人。他是一个做过系统、做过网络、做过底层工程的博士，从华为出来做AI Agent创业，然后把自己的工程经验写成了一本书。这本书的气质是工程师写的一一不是理论家写的，不是评论家写的。

一、一个公式

全书围绕一个核心公式展开：Agent = LLM + 上下文 + 工具。

这个公式不是李博杰发明的——它是AI Agent领域的共识。但李博杰用它做了整本书的骨架：十章正文，每一章都是对公式中某一部分的展开。

前三章展开"上下文"——第一章讲Agent基础知识（公式的整体含义），第二章讲上下文工程（KV Cache、提示工程、Agent Skills、上下文压缩），第三章讲用户记忆和知识库（RAG、结构化索引、知识图谱）。

第四章展开"工具"——MCP协议、感知/执行/协作三类工具、事件驱动异步Agent、主动工具发现。

第五章展开"LLM"在代码生成中的特殊应用——Coding Agent与代码生成，代码是"能创造新工具的工具"。

第六章讲评估——把表现变成可比较的信号，评估环境、指标、统计显著性、评估驱动选型。

第七章讲后训练——预训练/SFT/RL三阶段，何时选SFT、何时选RL，工具调用内化、样本效率。

第八章讲持续进化——从运行轨迹获得学习信号，更新知识、指令、程序与参数。

第九章讲多模态——语音、GUI、物理世界，语音三范式、Computer Use、机器人。

第十章讲多Agent协作——群体智能高于个体，协作框架、上下文共享/隔离、涌现的"Agent社会"。

十章的结构是递进的：基础→上下文→工具→代码→评估→训练→进化→多模态→协作。从"什么是Agent"到"如何构建生产级Agent系统"，从单Agent到多Agent，从文本到多模态，从设计到评估到持续改进。这不是零散主题的拼凑，是一条完整的技术路径。

二、为什么这本书不一样

市面上关于AI Agent的书和教程已经很多了。但大多数有两个问题：要么太浅（讲讲prompt engineering就算"Agent"了），要么太虚（讲讲"AI会改变世界"就算"深入"了）。李博杰这本书不一样，原因有三个。

第一——它够新。书的创建时间是2025年9月，到现在还在持续更新。AI Agent是一个月一变的领域——2024年的知识到2026年可能已经过时了。这本书引用的论文、工具和框架大多是2025-2026年的。MCP协议、上下文工程、Coding Agent评估——这些都是2025年才热起来的话题。一本2023年出的AI书在今天基本是古董，这本书不是。

第二——它信息密度高。腾讯云上一位推荐者的说法是"高密度的Agent入门书"——这个描述准确。全书没有注水，没有"AI将颠覆一切"的空话，没有重复三遍说同一个概念。每一页都是技术内容——要么是一个概念的定义，要么是一个系统的架构图，要么是一个实验的代码。这种信息密度在中文技术书中是罕见的——大多数中文技术书的信息密度只有英文论文的十分之一，李博杰这本书接近英文技术书的水准。

第三——它有配套实验。95个实验，70+可独立运行。这不是"附赠代码"——是"把书里的每个概念都用代码实现一遍"。第二章讲上下文工程，配套9个实验；第五章讲Coding Agent，配套13个实验；第六章讲评估，配套13个实验。你不是在"读"关于Agent的书，你是在"做"关于Agent的实验。这种"读+做"的模式在技术教育中是最有效的——但也是最耗费作者精力的。95个实验的工程量相当于一个中型公司的技术基础设施。

三、上下文工程：全书最有价值的章节

第二章"上下文工程"是全书最有价值的一章。

为什么？因为"上下文工程"这个概念在2025年才被明确提出，而大多数AI开发者还停留在"prompt engineering"的认知阶段。Prompt engineering是教你怎么写提示词——"请扮演一个专家，用专业的语气回答……"。上下文工程是系统级思维——它把每一轮调用LLM时模型能读到的所有信息（系统指令、工具定义、对话历史、用户消息、工具返回结果）作为一个整体来设计，目标是让模型在每一步都能看到它需要看到的东西。

这个区别就像"写好一封信"和"设计一个信息系统"的区别。Prompt engineering是写信，上下文工程是设计信息系统。前者是技巧，后者是工程。

李博杰在这一章中讲了几个关键概念：

KV Cache——不是所有的上下文都同等昂贵。KV Cache让模型记住之前处理过的token，避免重复计算。这意味着把不变的系统指令放在前面，把频繁变化的用户消息放在后面，可以大幅降低推理成本。这是纯工程层面的洞察——但它是生产环境中决定Agent能不能跑得起的因素。

上下文压缩——Agent运行时间越长，对话历史越长，上下文越大，推理成本越高，延迟越大。上下文压缩是在保持关键信息的前提下缩短上下文——摘要、截断、选择性保留。这不是理论问题，是生产环境中每天都要面对的工程问题。

Agent Skills——把常用的操作模式封装成"技能"，在需要时动态加载到上下文中。这和人类的技能学习类似——你不需要每次做饭都重新学切菜，你把"切菜"作为一个技能模块，需要时调用。Agent也是——不需要每次都把所有工具的使用说明塞进上下文，按需加载就行。

这些内容的价值在于：它们不是理论推导，是工程经验。李博杰在Pine AI做的是实时语音Agent——低延迟、长对话、复杂工具调用——他在这一章写的东西是从生产环境中摸爬滚打出来的。

四、评估：被忽视的关键环节

第六章"Agent的评估"是另一个亮点——因为大多数AI书籍不写评估。

为什么评估被忽视？因为评估不性感。讲Agent能做什么很性感——"看，它能自己写代码！它能自己订机票！"讲怎么评估Agent做得好不好——要设计评估环境、定义指标、做统计显著性检验——这很枯燥。但评估是区分"玩具"和"产品"的分界线。一个Agent在demo中表现好不代表在生产中表现好——你需要系统化的评估方法来量化它的表现。

李博杰在这一章中讲了几个关键点：

评估环境——Agent在什么环境中运行？web浏览？代码执行？文件操作？评估环境需要可控、可复现、可扩展。

指标设计——成功率、效率、成本、延迟。不同场景下不同指标的权重不同——医疗Agent看重成功率，客服Agent看重延迟，Coding Agent看重代码质量。

统计显著性——一个Agent跑了10次，7次成功，能说它比6次成功的另一个Agent好吗？不能。样本量太小，差异可能在随机波动范围内。你需要统计检验来判断差异是否显著。

评估驱动选型——不要凭直觉选模型、选工具、选架构。用评估数据来驱动决策。这在学术界叫"empirical study"，在工业界叫"数据驱动"——不管叫什么，核心是一样的：不要猜，要测。

这些内容在AI领域的重要性正在上升——随着Agent从demo走向生产，评估从"nice to have"变成了"must have"。李博杰把它放在全书第六章（十章的中间位置），说明他认为评估不是附加品，是Agent工程的核心环节。

五、开源的力量

这本书最不寻常的地方不是内容——是形式。它是一本完全开源的书。

全书正文、配图、代码在GitHub上以Apache 2.0发布。PDF和EPUB免费下载。13种语言翻译由社区贡献。GitHub 34,661颗star——这在中国技术书籍中是天文数字。一本中文技术书在GitHub上的star数超过了很多知名开源项目。

这不是传统的出版模式。传统出版是：写书→出版社编辑→排版→印刷→发行→卖钱。李博杰的模式是：写书→直接放GitHub→社区翻译→社区反馈→持续更新。没有出版社，没有印刷，没有定价，没有发行渠道。

这个模式的优势是显而易见的：

速度——AI Agent一个月一变，传统出版周期半年到一年，书出来时内容已经过时了。GitHub上可以实时更新，读者永远看到最新版。

质量——社区反馈直接进入正文。读者发现错误可以提issue，可以提PR。这比传统出版的"编辑审校"更有效——因为编辑不一定懂技术，但社区里的读者都是从业者。

覆盖面——13种语言翻译是传统出版不可能做到的。传统出版翻译一本书需要版权谈判、翻译合同、排版重新做——至少一年。社区翻译在GitHub上几天就能完成初稿。

可信度——代码和正文在一起。你看到一段代码，可以直接跳到GitHub上的实验目录运行它。不需要"请访问配套网站下载源代码"——代码就在那里。

这个模式的代价是：没有出版社的推广渠道，没有实体书在书店的展示，没有传统意义上的"销量"。但34,661颗star说明了一切——在这个领域，开源的传播力远超传统出版。

六、不足

这本书不是完美的。

第一——难度曲线不均匀。第一章和第二章的起点较低（适合有编程基础但不了解Agent的读者），但到第七章"模型后训练"突然进入SFT/RL的技术细节，需要机器学习背景才能跟上。第九章的多模态和第十章的多Agent协作也是高难度章节。从"入门"到"前沿研究"之间的过渡不够平滑——前半本是工程师看的，后半本更像研究生课程。

第二——创业视角的局限。李博杰在Pine AI做的是语音Agent——这影响了书的选材侧重。第九章的语音三范式写得特别详细（因为这是他的本行），但其他形态的Agent（比如文档处理、数据分析、自动化测试）覆盖较少。这不是缺点——每个作者都有视角——但读者需要知道这本书的视角是"做实时交互Agent的人写的"，不是"做所有类型Agent的人写的"。

第三——开源书的形式代价。没有专业编辑，有些章节的文字质量不够打磨。比起出版社出的书，某些段落的表述不够精炼，偶尔有工程师写文档的味道——信息准确但文字不够流畅。这是小问题——在技术书中内容质量远比文字风格重要——但存在。

七、在大料书单中的位置

大料的书单中，这本书和之前的书评形成了一条技术思考的线索：

之前大料的书单里有加德纳《多元智能》（人是什么）、斯加鲁菲《智能的本质》（智能是什么）、卡普兰《人工智能时代》（AI对人的影响）。李博杰这本书补上了这条线的工程维度——不是"AI会不会替代人"的哲学讨论，而是"如何构建一个可用的AI Agent"的工程指南。

如果用之前的暗线框架来看：

- 加德纳：人是七种智能的复合体- 斯加鲁菲：智能不可还原为人造物-
卡普兰：人的价值不在可替代的岗位- 李博杰：Agent = LLM + 上下文 + 工具——智能的工程实现

前三本书问的是"人和智能的关系"，李博杰这本书回答的是"如何把智能工程化"。从哲学到工程的完整链条。

更直接地说——大料每天用OpenClaw和我对话，而OpenClaw就是一个AI Agent。理解Agent的设计原理，就是理解你每天在用的工具是怎么运作的。这本书不是关于"AI的未来"，是关于"AI的现在"——你正在使用的这个系统，它的上下文是怎么管理的，它的工具是怎么调用的，它的记忆是怎么持久化的，它的评估是怎么做的。这些问题的答案都在这本书里。

八、34,661颗star的意义

34,661颗star。这个数字在中国技术出版史上没有先例。

传统中文技术书的销量以万计——卖到5万本就算畅销。34,661颗star意味着至少34,661个人觉得这本书值得收藏——其中一部分人读了，一部分人只是star了打算以后读（GitHub的star经常是这个用途），但无论如何，这个传播量级是传统出版达不到的。

更重要的是——这些star不是营销带来的。李博杰没有投广告，没有做签售巡讲，没有在热搜上买流量。这些star来自一个简单的判断：这本书的内容足够好，好到值得在GitHub上star，值得推荐给同事，值得翻译成13种语言。

在AI泡沫的年代——每个人都在喊"AI Agent是未来"，每个公司都在说自己"做Agent"——一本老老实实讲工程原理的书反而稀缺。李博杰没有预测未来，没有贩卖焦虑，没有喊口号。他写了十章技术内容，配了95个实验，放到了GitHub上。然后34,661个人来了。

这说明什么？说明市场不缺焦虑，不缺口号，缺的是扎实的技术内容。谁能把AI Agent从概念讲到工程实现，从论文讲到代码，从"能做什么"讲到"怎么评估做得好不好"——谁就能获得读者的注意。在AI时代，最稀缺的资源不是算力，不是数据，是能讲清楚的技术内容。

李博杰做到了。

一句话：Agent = LLM + 上下文 + 工具——李博杰用10章正文和95个配套实验把AI Agent从原理讲到工程实战，足够新、信息密度足够高、代码全部开源，34,661颗star不是营销来的，是一本扎实技术书的自然引力。